

# 完整執行紀錄

從乾淨狀態跑完整條流程，每個階段都是「指令 → 程式碼在做什麼 → 實際輸出」三段式。

## 這份文件怎麼讀

追蹤檔的資料量很大——單一個 `train_step_0001.txt` 就有 **926 行 / 59 KB**。全部貼出來沒有意義，所以這裡採三層呈現：

| 層  | 做什麼                  | 在哪            |
|----|----------------------|---------------|
| 摘要 | 一張表說明產出了什麼檔案、多大、要看什麼 | 產出的檔案         |
| 骨架 | 長檔案的章節目錄與行號，知道去哪裡找   | 怎麼讀 926 行的追蹤檔 |
| 精選 | 5 個關鍵片段，配上對應的程式碼     | 下面每個階段        |

原則：這份文件負責告訴你「該看哪幾行」，不負責取代那些檔案。

## 環境

|        |                                                |
|--------|------------------------------------------------|
| GPU    | NVIDIA RTX PRO 6000 Blackwell (96 GB)          |
| Python | 3.12, torch 2.9.1+cu130                        |
| 語料     | <code>data/corpus_zhtw.jsonl</code> 前 20,000 行 |
| 總耗時    | 梯度驗證 3 秒 + BPE 訓練 12 秒 + 模型訓練 79 秒             |

```
rm -rf out traces          # 從乾淨狀態開始
```

## 階段 1：驗證所有手推的公式

```
python tests/test_layers.py
```

### 程式碼在做什麼

不用 PyTorch 的 autograd 對答案，而是回到梯度的**定義**直接量：

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w - \varepsilon)}{2\varepsilon}$$

```
# scratch/gradcheck.py (節錄)
y0 = layer.forward(x)
dy = torch.randn(y0.shape)          # 隨機的上游梯度
dx = layer.backward(dy)              # 手推公式算出來的

flat[i] = orig + eps; lp = float((layer.forward(x) * dy).sum())
flat[i] = orig - eps; lm = float((layer.forward(x) * dy).sum())
num = (lp - lm) / (2 * eps)          # 有限差分算出來的
rel = abs(num - float(dx.reshape(-1)[i])) / ...
```

人工造一個純量 loss  $L = \sum(y \odot dy)$ ，這樣  $\partial L / \partial y$  剛好等於  $dy$ ，就能用任意的  $dy$  去測 backward 對不對。

用**中央差分**而不是單邊，因為誤差是  $O(\epsilon^2)$  而不是  $O(\epsilon)$ 。整個檢查跑在 `float64` 上，避免 `float32` 的捨入誤差蓋過訊號。

## 實際輸出

梯度檢查 (有限差分 vs 手推公式)

```
=====
Linear 無bias          最大相對誤差 2.29e-12  PASS
Linear 有bias          最大相對誤差 2.03e-12  PASS
RMSNorm                最大相對誤差 2.13e-07  PASS
Embedding (scatter-add) 最大相對誤差 6.80e-14  PASS
TiedOutputHead         最大相對誤差 5.61e-12  PASS
SwiGLU                 最大相對誤差 1.69e-06  PASS
rope                   最大相對誤差 2.19e-12  PASS
Attention               最大相對誤差 3.32e-07  PASS
Softmax+CrossEntropy   最大相對誤差 7.50e-08  PASS
=====
最差相對誤差 1.69e-06  -> 全部通過
```

誤差大小的差異是有意義的：純矩陣運算的層（Linear、Embedding、RoPE）在  $10^{-12}$  等級，含開根號、除法、 $\sigma(1-\sigma)$  的層（RMSNorm、SwiGLU）在  $10^{-6} \sim 10^{-7}$ ，因為浮點誤差會累積。

## 階段 2：訓練 BPE tokenizer

```
python train.py --tokenizer bpe --vocab 6400 ...
```

## 程式碼在做什麼

$$|\mathcal{V}| = |\mathcal{V}_0| + M$$

反覆合併最高頻的相鄰配對，每合併一次詞表 +1，到目標大小就停。

```
# tokenizer/bpe.py (節錄)
for step in range(n_merges):
    pair, cnt = max(pair_cnt.items(), key=lambda kv: kv[1]) # 最高頻的配對
    new_sym = pair[0] + pair[1]
    for i in pair_where[pair]: # 只更新受影響的詞塊
        deindex_word(i)
        ...把 pair 換成 new_sym...
        index_word(i)
    tk.merges.append(pair)
    tk.itos.append(new_sym) # 詞表 +1
```

`pair_where` 記錄「哪些詞塊含有這個配對」，合併時只更新受影響的那些，不用每輪重掃整份語料——否則 1,173 次合併會跑很久。

## 實際輸出

```
詞塊 1,826,987 個（相異 599,629 個），字元 5,415,807 個
起始詞表 5,227（4 特殊 + 5,223 字元）
目標 6,400 → 需要合併 1,173 次

合併 #1      '可' + '以' -> '可以'（23,763 次）  詞表 5,228
合併 #1000   '生' + '產' -> '生產'（393 次）   詞表 6,227

完成，最終詞表 6,400
```

$5,227 + 1,173 = 6,400$ 。這就是「詞表 6400 哪來的」的答案。

注意合併次數一路遞減： $23,763 \rightarrow 393$ 。這給了一個實用判斷——如果最後幾次合併降到個位數，代表詞表開太大了。

完整的合併紀錄：`traces/bpe_training.txt`（54 行）

## 階段 3：訓練模型

### 模型規格

#### 模型結構

|                          |           |  |
|--------------------------|-----------|--|
| =====                    |           |  |
| embedding                | 1,638,400 |  |
| block0                   | 778,752   |  |
| block1                   | 778,752   |  |
| block2                   | 778,752   |  |
| block3                   | 778,752   |  |
| final_norm               | 256       |  |
| lm_head (與 embedding 共用) | 0         |  |
| -----                    |           |  |
| 總計                       | 4,753,664 |  |
| embedding 佔比             | 34.5%     |  |

optimizer 拿到 38 個參數張量

`lm_head` 是 0，因為它跟 embedding 共用同一個張量（weight tying）。`optimizer` 拿到 38 個張量——**embedding 就在這 38 個裡面**，跟 `q_proj` 平起平坐。

### 訓練迴圈

```
# train.py (節錄)
model.zero_grad()
logits = model.forward(x, tr)      # 前向
loss   = ce.forward(logits, y, tr) # 算 loss
dlogits = ce.backward(tr)          # dz = (p - y) / N
model.backward(dlogits, tr)         # 反向，逐層往下

gnorm = opt.clip_grad_norm(args.clip) # 全域梯度裁剪
opt.step(tr, watch='embed.E')        # 這一行才真的改動權重
```

**注意** `model.backward()` 和 `opt.step()` 是兩件事。前者只算出「該往哪修」存進梯度表，後者才動權重。

## loss 曲線

| step | loss          | lr        | 梯度範數   | 秒    |
|------|---------------|-----------|--------|------|
| 1    | 8.8149        | 3.000e-05 | 0.8127 | 0.4  |
| 200  | 5.3237        | 2.992e-03 | 0.3945 | 5.7  |
| 500  | 4.2052        | 2.875e-03 | 0.3919 | 13.6 |
| 1000 | 3.6166        | 2.408e-03 | 0.3512 | 26.8 |
| 1500 | 3.1605        | 1.723e-03 | 0.3512 | 40.0 |
| 2000 | 3.0630        | 1.018e-03 | 0.3594 | 53.1 |
| 2500 | 2.9718        | 4.932e-04 | 0.3764 | 66.2 |
| 3000 | <b>2.7806</b> | 3.000e-04 | 0.3886 | 79.4 |

8.81 → 2.78，79 秒。lr 的形狀是 warmup（前 100 步爬升）接 cosine 衰減。

完整曲線每 20 步一筆：`traces/train_log.txt`（152 行）

## 階段 4：生成

```
python generate.py --prompt "機器學習是"
```

### 實際輸出

機器學習是一項重要的領域，即將資料能夠從大量資料中學習和推理。  
機器學習可以用於學習、自然語言處理和自然語言生成等各種任務。

台灣的天氣如今，在這裡有著令人無法抗衰老的環境和繁華性災難。  
這些災難事件可以用於對當地居民群落和探討。

貓是一種常見的寵物，並且常常會受到許多人關注和感悟。  
狗通常很快地奔跑、游泳、跳傘……

第一段語意通順、術語正確。後兩段語法沒問題但語意會飄——這就是 475 萬參數、吃 424 萬 token 該有的水準。

（機器學習 被 BPE 合併成單一 token，所以模型對它的處理特別穩。）

## 階段 5：權重與 tokenizer 綁定

```
python tests/test_tokenizer_binding.py
```

### 實際輸出

① 正確的 tokenizer                      loss = 3.5754  
② id 對應被打亂                      loss = 12.0569

換算成 perplexity（等效於在幾個選項中亂猜）：

正確 tokenizer        35.7

打亂之後            : 172287.4

完全隨機的模型    : 6400.0

③ 詞表大小不同 -> RuntimeError: size mismatch（安全，馬上知道出事）

④ 擴充 6400 -> 6450，同一句話 loss 幾乎不變

用錯 tokenizer 比完全沒訓練過的模型還糟 27 倍，而且不會報錯。所以 `model.save()` 會把 tokenizer 的指紋一起存進去，載入時比對。

## 產出的檔案

| 檔案                                  | 行數  | 大小      | 看什麼                      |
|-------------------------------------|-----|---------|--------------------------|
| <code>gradcheck.txt</code>          | 18  | 0.9 KB  | 九層的驗證總表，可直接貼進簡報          |
| <code>model_summary.txt</code>      | 32  | 1.1 KB  | 參數量分佈與所有超參數              |
| <code>bpe_training.txt</code>       | 54  | 4.0 KB  | 詞表怎麼從 5,227 長到 6,400     |
| <code>train_log.txt</code>          | 152 | 8.6 KB  | 每 20 步的 loss / lr / 梯度範數 |
| <code>inference.txt</code>          | 456 | 28.6 KB | 一次推論的完整前向（沒有反向）          |
| <code>train_step_0001.txt</code>    | 926 | 58.8 KB | 第 1 步的完整前向 + 反向 + 更新     |
| <code>train_step_3000.txt</code>    | 925 | 58.8 KB | 最後一步，可跟第 1 步對照           |
| <code>out/model.pt</code>           | —   | 18.6 MB | 訓練好的權重                   |
| <code>out/tokenizer_bpe.json</code> | —   | 65 KB   | 詞表與合併規則                  |

## 怎麼讀 926 行的追蹤檔

不要從頭讀。先看骨架，再跳到想看的地方。

### 骨架（行號可直接跳）

```
8  這一批的資料          ← 這批文字長什麼樣
19 前向傳播
25  embed · forward (查表)
45  ===== block0 ===== ← 每個 block 約 100 行
474 softmax + cross-entropy · forward
493 softmax + cross-entropy · backward ← p - y 在這裡
510 反向傳播
776  ===== block0 · backward =====
851  embed · backward (scatter-add) ← 兩條梯度路徑的拆解
889 AdamW 更新（第 1 步）      ← 權重真正被改的地方
917 這一步結束
```

`block1` / `block2` / `block3` 的結構跟 `block0` 一模一樣，看一個就夠。

### 導覽：五個最值得看的片段

#### ① 第 25 行 — 查表就是查表

公式：  $x = E[\text{ids}]$

等價於  $x = \text{one\_hot}(\text{ids}) @ E$ ，只是跳過了乘以 0 的部分

`ids`                      `shape (64, 128)`

`x`                        `shape (64, 128, 256)`

| 位置  | 查哪一列    | E[id] 的前 4 維 |          |          |          |
|-----|---------|--------------|----------|----------|----------|
| t=0 | id=4    | +0.01647     | -0.04764 | +0.02504 | +0.00472 |
| t=1 | id=5868 | -0.00623     | +0.02665 | -0.00391 | -0.01012 |

>> 這一批用到 2036 個不同的 token，詞表共 6400 個。

>> 剩下 4364 列這一步完全沒被碰到——反向時它們的梯度會是 0。

前向沒有任何乘法，就是索引。記住 `2036 / 6400` 這個數字，第 ④ 點會用到。

#### ② 第 493 行 — 整條鏈唯一知道答案的地方

公式:  $dz = (p - y) / N$

| token id | p        | y        | $dz = (p-y)/N$ | 作用      |
|----------|----------|----------|----------------|---------|
| 4656     | +0.00048 | +0.00000 | +5.920e-08     | 壓下去     |
| 220      | +0.00046 | +0.00000 | +5.575e-08     | 壓下去     |
| 5868     | +0.00026 | +1.00000 | -0.00012       | 拉上來（正解） |

正解那一格  $y = 1$ ，所以  $dz$  是**負的**。更新時  $E \leftarrow E - \eta dz$ ，負負得正，分數被推高。其他格全是正的，分數被壓低。

這一行就是整個訓練的全部監督訊號。往下每一層都只是機械式的微分。

### ③ 第 851 行 — scatter-add，以及它不是唯一的來源

公式:  $dE[v] += \sum_{\{t : id\_t = v\}} dx[t]$

這一系列的梯度其實是兩條路加起來的：

|                     | 來源 | $dE[5][0]$ | 有幾列非零            |
|---------------------|----|------------|------------------|
| 輸入側（本步 scatter-add） |    | +0.00125   | 2036 / 6400 ← 稀疏 |
| 輸出側（weight tying）   |    | -0.00420   | 6400 / 6400 ← 稠密 |
| 合計 $dE[5][0]$       |    | -0.00295   | 6400 / 6400      |

2036 / 6400 正是第 ① 點那個數字——輸入側只碰得到出現過的 token。但因為  $E$  同時是分類器權重，softmax 會碰到**每一列**，所以總梯度是稠密的。

注意兩條路的符號還相反（+0.00125 vs -0.00420），合計後變號。

### ④ 第 889 行 — 位移恰好等於 $lr$

|                                     |            |            |            |            |
|-------------------------------------|------------|------------|------------|------------|
| 更新前 p                               | +0.00388   | +0.04323   | -0.00344   | +0.01698   |
| 梯度 g                                | -1.290e-06 | +8.668e-05 | -5.620e-05 | -6.518e-06 |
| $\hat{m}$                           | -1.290e-06 | +8.668e-05 | -5.620e-05 | -6.518e-06 |
| $\sqrt{\hat{v}}$                    | +1.290e-06 | +8.668e-05 | +5.620e-05 | +6.518e-06 |
| $\hat{m}/(\sqrt{\hat{v}}+\epsilon)$ | -0.99231   | +0.99988   | -0.99982   | -0.99847   |
| 位移 $\Delta$                         | +2.977e-05 | -3.000e-05 | +2.999e-05 | +2.995e-05 |
| lr                                  |            |            |            | 3.000e-05  |

看第 2 行和第 4 行：梯度從  $1.29 \times 10^{-6}$  到  $8.67 \times 10^{-5}$ ，**差了 67 倍**。但位移全都是  $\pm 3 \times 10^{-5}$ ，**一模一樣**。

因為第一步時  $\hat{m} = g$ 、 $\sqrt{\hat{v}} = |g|$ ，相除必定是  $\pm 1$ ：

$$\frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = \frac{g}{|g|} = \text{sign}(g)$$

Adam 讓所有參數走一樣遠，差別只在方向。

#### ⑤ 對照 train\_step\_3000.txt

同一個位置的數字，訓練 3000 步之後：

```
diff <(sed -n '493,500p' traces/train_step_0001.txt) \  
    <(sed -n '493,500p' traces/train_step_3000.txt)
```

第 1 步時正解的機率是 0.00026（幾乎是 1/6400 的隨機猜測），第 3000 步會明顯高出來。這是最直接的「模型學到東西了」的證據。

## 重跑

```
rm -rf out traces  
python tests/test_layers.py  
python train.py --tokenizer bpe --vocab 6400 --steps 3000 \  
    --dim 256 --layers 4 --heads 4 --seq 128 --batch 64 \  
    --trace-steps 1,3000  
python generate.py --prompt "機器學習是"  
python tests/test_tokenizer_binding.py
```

固定 seed（預設 42），所以數字可以完全重現。